

UR5e Control Analysis

Part 1B-4 — Model Predictive Control Design, Computational Optimization, and Nominal Benchmark

Sooyong Kim
Robotics Engineering Portfolio Project

September 2026

Abstract

This report documents the design, tuning, computational restructuring, nonlinear nominal benchmark, and Python–C++ cross-validation of a constrained linear model predictive controller (MPC) for the validated UR5e rigid-body model established in Part 1A. The controller uses one fixed discrete linearization at the nominal start configuration, exact zero-order-hold discretization at a 2 ms sample time, a prediction horizon of 160 samples (320 ms), a control horizon of 40 samples (80 ms), and explicit joint-torque constraints. Reference-dependent gravity feedforward is incorporated as a known affine input shift.

The final nominal controller achieved an overall joint RMSE of 0.048474° , TCP position RMSE of 0.378424 mm, and TCP orientation RMSE of 0.129772° . Peak torque utilization was 27.17% on Joint 2; no nominal sample activated the adopted torque constraints. The resulting benchmark therefore measures predictive tracking behavior rather than saturation recovery.

The central engineering issue was computational rather than feasibility. The initial Python implementation required approximately 17.97 ms per online update, far above the 2 ms control period. Profiling showed that repeated dense gradient construction and horizon assembly dominated the runtime. By moving fixed matrix products and known reference-dependent terms offline, vectorizing horizon construction, and selecting OSQP tolerances based on measured solution sensitivity, the mean Python online time was reduced to approximately 1.08 ms without changing the reported tracking result. A C++/OSQP implementation then reproduced the Python torque sequence with a maximum difference of 2.03×10^{-10} N m and achieved a mean online computation time of 0.960 ms with a 0.973 ms P99. The observed maximum timing outlier remained approximately 9.95 ms, so these measurements establish practical average/P99 computational feasibility for a 500 Hz simulation benchmark, not hard real-time determinism.

1 Scope and Benchmark

Part 1A established the nonlinear torque-driven UR5e plant and the common 4 s quintic joint-space benchmark. The reference moves from

$$q_0 = [0, -90, 90, -90, -90, 0]^\circ$$

to

$$q_f = [20, -60, 60, -70, -70, 20]^\circ$$

with zero endpoint velocity and acceleration. The sample period is $T_s = 0.002$ s, giving 2001 samples. The adopted simulation torque constraints are

$$|\tau_{1,2,3}| \leq 150 \text{ N m}, \quad |\tau_{4,5,6}| \leq 28 \text{ N m}.$$

As in the preceding controller reports, these are simulation constraints inherited from the imported model, not independently verified physical motor ratings.

The purpose of this stage was to answer four questions: whether a local linear predictive model can track the nonlinear benchmark; how prediction and control horizon choices trade tracking against computation; whether explicit torque constraints become active under nominal operation; and whether the resulting algorithm can be restructured into an online form suitable for later Isaac Sim validation.

2 MPC Formulation

The nonlinear plant is

$$M(q)\ddot{q} + c(q, \dot{q}) + g(q) = \tau, \quad x = [q^T \dot{q}^T]^T.$$

I used the same fixed nominal equilibrium and local continuous linearization used for the LQR study,

$$\delta\dot{x} = A\delta x + B\delta\tau,$$

then obtained (A_d, B_d) by exact zero-order-hold discretization at 2 ms. With

$$z_k = x_k - x_{\text{eq}}, \quad v_k = \tau_k - \tau_{\text{eq}},$$

the prediction model is

$$z_{k+1} = A_d z_k + B_d v_k.$$

The trajectory-dependent feedforward torque is introduced as

$$\tau_i = g(q_{\text{ref},i}) + w_i,$$

so that

$$v_i = [g(q_{\text{ref},i}) - \tau_{\text{eq}}] + w_i.$$

This distinction matters: the optimizer acts on the correction w_i , while the known gravity shift varies along the reference.

The quadratic program is written in the standard form

$$\min_W \frac{1}{2} W^T P W + q^T W$$

subject to reduced joint-torque bounds. The final design uses

$$N_p = 160, \quad N_c = 40, \quad \rho_u = 10.$$

After N_c , the final correction is held through the remainder of the prediction horizon. Consequently, the last reduced control bound is the intersection of all torque-feasible bounds over the held tail, rather than the bound at only one prediction step.

3 Prediction-Horizon Study

I first increased the prediction horizon while retaining the full control horizon. Longer prediction substantially reduced the final Joint 2 error, but the original dense implementation incurred rapidly increasing solve cost.

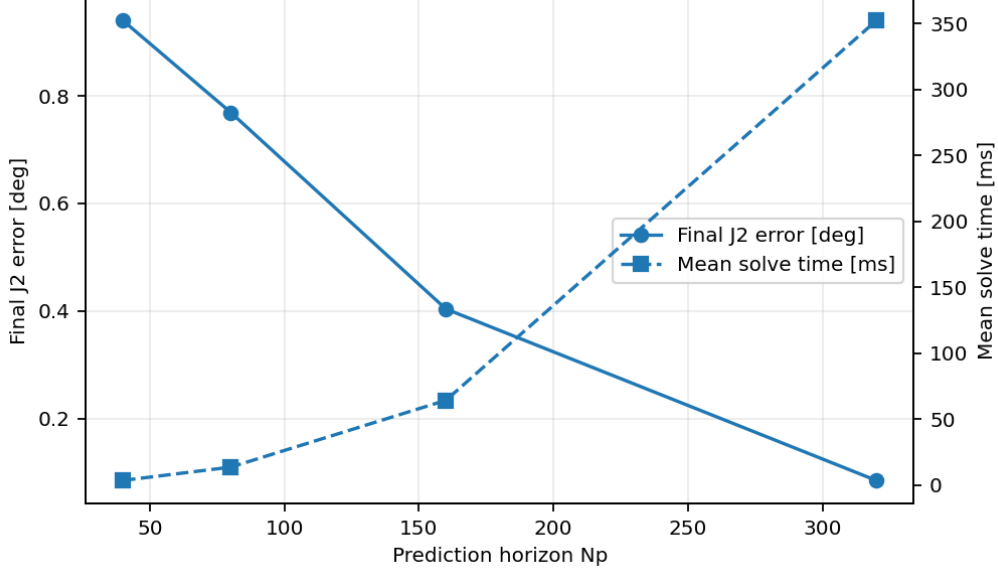


Figure 1: Prediction-horizon sweep at $\rho_u = 10$. Longer look-ahead improves the tested terminal tracking metric but sharply increases the cost of the original dense implementation.

The tested $N_p = \{40, 80, 160, 320\}$ cases produced final Joint 2 errors of approximately $\{0.940, 0.769, 0.404, 0.085\}$. I did not select $N_p = 320$ simply because it produced the smallest tested terminal error; its original mean solve time was approximately 352 ms. Instead, $N_p = 160$ was retained as a meaningful prediction window for subsequent control-horizon and implementation studies.

4 Control-Horizon Reduction

The control horizon was then shortened while holding $N_p = 160$. Reducing N_c from 160 to 40 increased the tested final Joint 2 error only from approximately 0.4037° to 0.4102° in the horizon study, while the original mean solve time fell from approximately 13.86 ms to 1.06 ms.

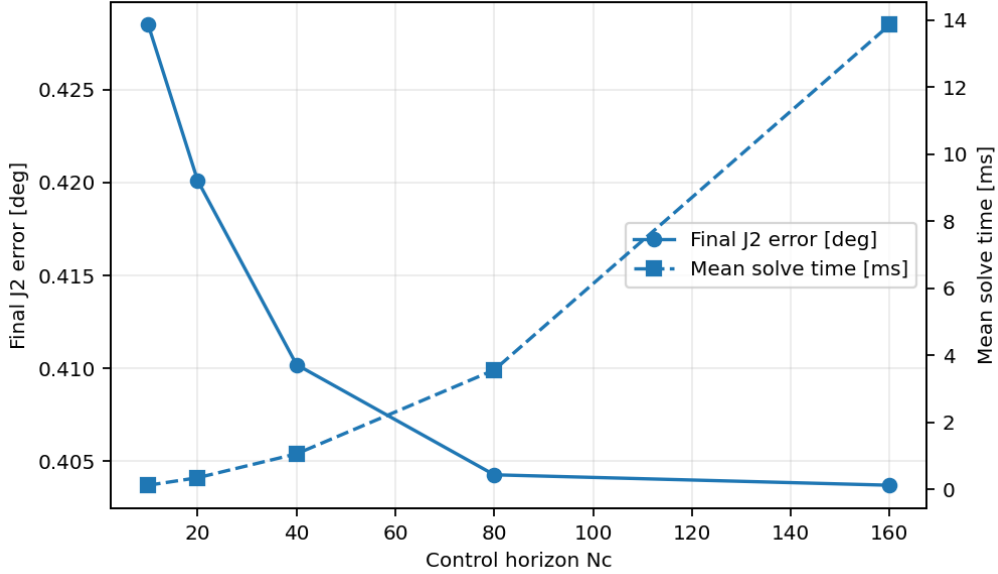


Figure 2: Control-horizon study with $N_p = 160$. $N_c = 40$ retained most of the tested predictive benefit while greatly reducing optimization dimension and solve cost.

I therefore selected $N_c = 40$, corresponding to an 80 ms independently optimized correction sequence followed by a held terminal correction over the remaining prediction window.

5 OSQP Tolerance Study

The first constrained implementation used absolute and relative OSQP tolerances of 10^{-8} . I then tested 10^{-6} and 10^{-5} using the same optimized MPC structure.

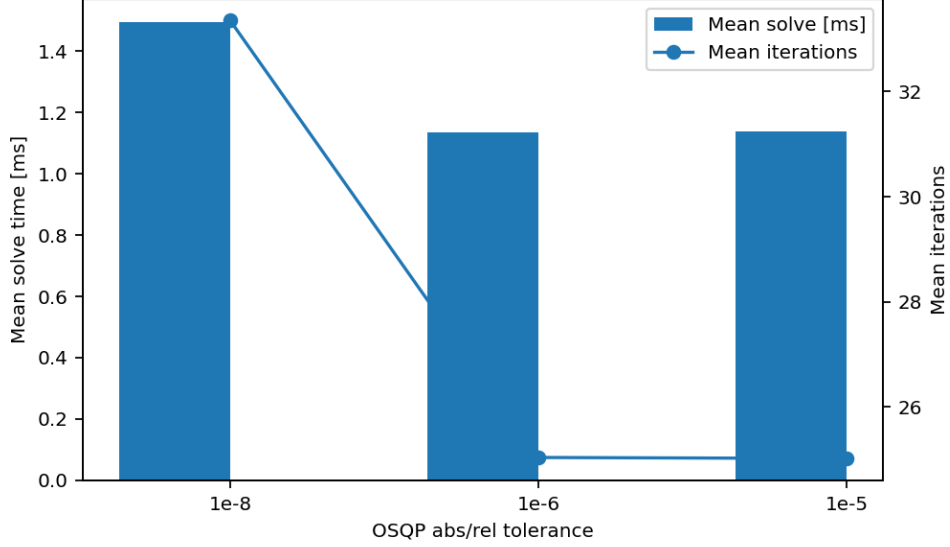


Figure 3: OSQP tolerance study. Relaxing the tolerance from 10^{-8} to 10^{-6} reduced the mean iteration count from about 33.35 to 25.04 with negligible first-move torque change.

At 10^{-6} the maximum tested first-move torque difference relative to the 10^{-8} solution was only 2.25×10^{-5} N m. Relaxing further to 10^{-5} did not materially reduce the mean iteration count or solve time, while increasing the maximum first-move difference to 4.14×10^{-4} N m. I therefore retained 10^{-6} for both absolute and relative tolerances.

6 Online Computational Restructuring

The first nominal Python benchmark required approximately 17.97 ms per update. Profiling separated horizon construction, state prediction, QP-gradient construction, bounds, OSQP update, and solve time. The dominant avoidable cost was repeated evaluation of algebra that was fixed for the chosen controller and known reference.

The original online gradient had the form

$$q_k = 2G_c^T \bar{Q} (Fz_k + GV_{ff,k} - Z_{ref,k}).$$

Defining

$$K_x = (2G_c^T \bar{Q})F$$

and precomputing

$$b_k = (2G_c^T \bar{Q}) (GV_{ff,k} - Z_{ref,k})$$

for the known nominal trajectory reduces the online calculation to

$$q_k = K_x z_k + b_k.$$

This is an algebraic decomposition of the same nominal QP; it does not change the horizon, weights, constraints, or reported control result.

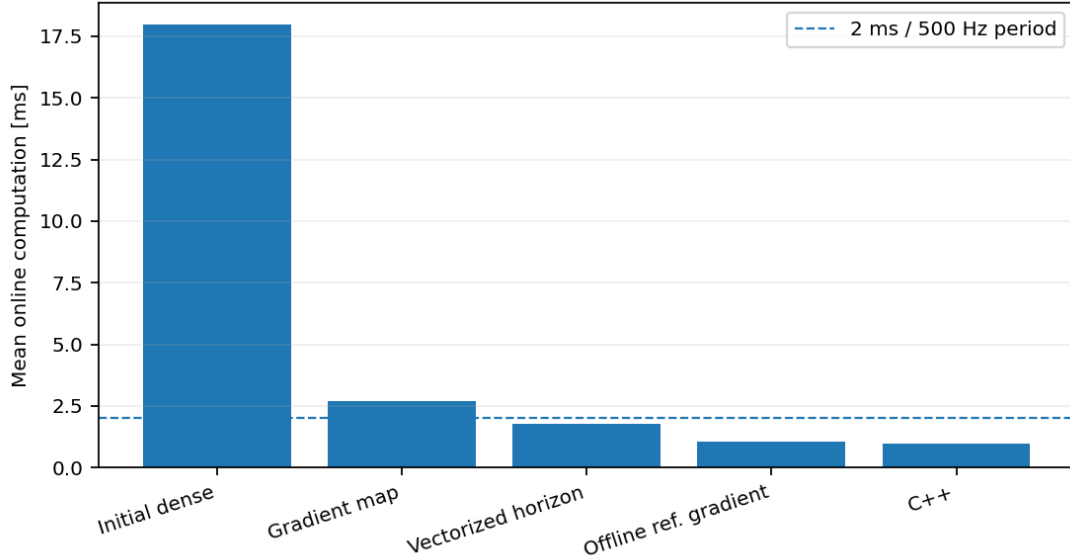


Figure 4: Measured mean online computation through the implementation sequence. The 2 ms line represents the 500 Hz benchmark period.

The progression from the initial dense Python implementation to the final C++ implementation reduced mean online computation from approximately 17.97 ms to 0.960 ms, a reduction of about 94.7%. The key engineering result is not merely that C++ is faster: most of the reduction occurred by identifying which calculations belonged offline rather than online.

7 Final Nominal Nonlinear Benchmark

The final MPC was evaluated on the same nonlinear RK4/ZOH plant used for the other Part 1B controllers. The final joint error metrics were:

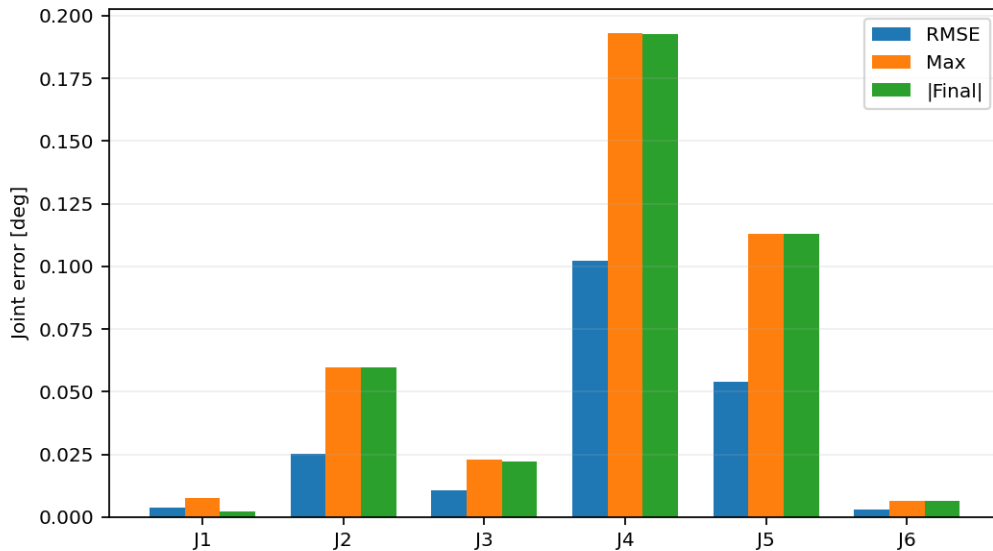


Figure 5: Per-joint RMSE, maximum error, and absolute final error for the final nominal MPC benchmark.

Table 1: Final nominal joint tracking metrics.

	J1	J2	J3	J4	J5	J6
RMSE [deg]	.00385	.02525	.01059	.10212	.05380	.00315
Max [deg]	.00748	.05978	.02295	.19299	.11296	.00642
Final [deg]	.00241	.05978	-.02228	.19269	-.11296	.00642

The overall joint RMSE was 0.048474° . The TCP position RMSE/max/final values were $0.378424/0.817053/0.817053$ mm, while orientation RMSE/max/final values were $0.129772/0.258784/0.258784^\circ$.

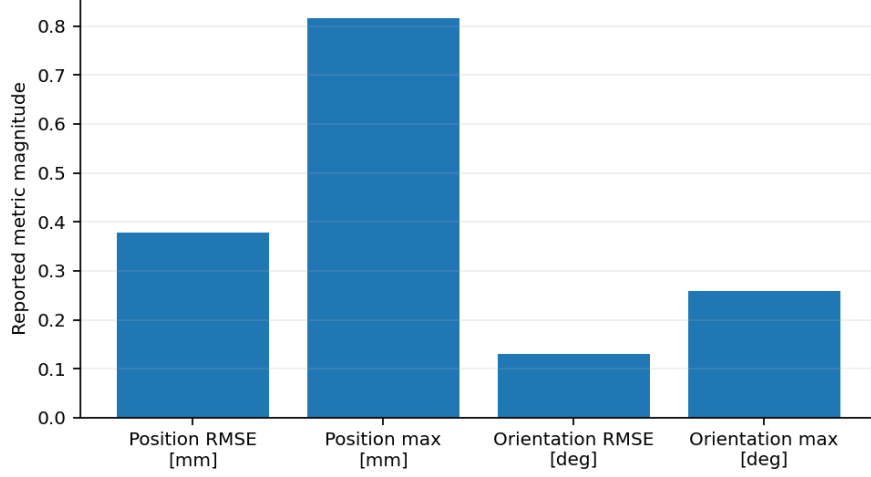


Figure 6: Reported TCP position and orientation tracking metrics for the final nominal benchmark. Position and orientation use different physical units and are shown only as a compact summary of the reported values.

8 Control Effort and Constraint Activity

The maximum torque vector was

$$[0.2074, 40.7606, 20.2635, 1.82528, 0.18149, 1.26 \times 10^{-5}] \text{ N m.}$$

Joint 2 again carried the dominant nominal demand.

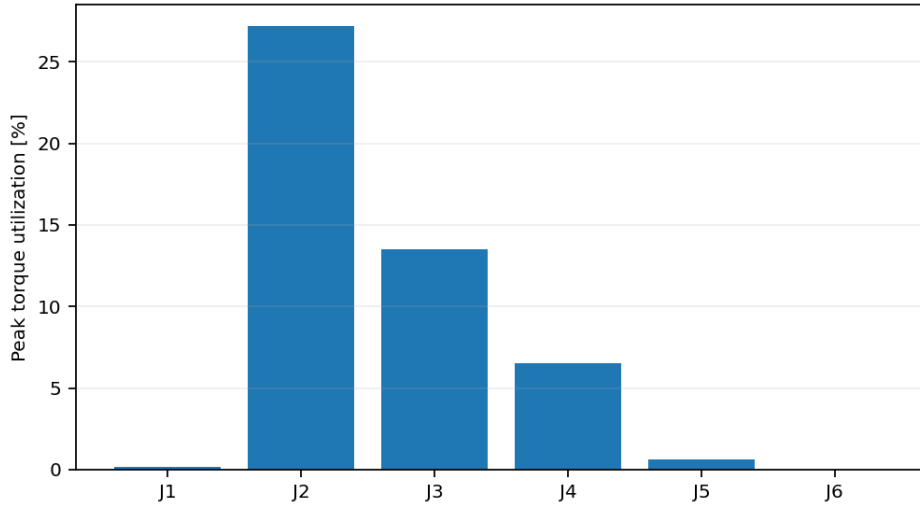


Figure 7: Peak torque utilization relative to the adopted simulation constraints.

No sample was saturated and no MPC torque constraint became active in the nominal benchmark. This is an important interpretation boundary. The controller includes constraints correctly, but the nominal trajectory does not demonstrate behavior under active constraints. Constraint handling should therefore be assessed separately under a more demanding or perturbed condition rather than inferred from the nominal result.

9 Python–C++ Cross-Validation and Runtime

The final online form was reproduced in C++ using Eigen and OSQP. I exported the fixed QP Hessian, state-gradient map, precomputed reference-gradient sequence, feedforward torque sequence, and the complete 2001-sample nonlinear Python state history. The C++ controller was then driven by the exact same state sequence and its torque output was compared sample by sample with the Python reference.

The maximum absolute torque difference was

$$\max |\tau_{C++} - \tau_{Python}| = 2.03 \times 10^{-10} \text{ N m},$$

occurring at sample 1748 on Joint 3. Mean OSQP iterations were 25.0375, matching the Python benchmark behavior.

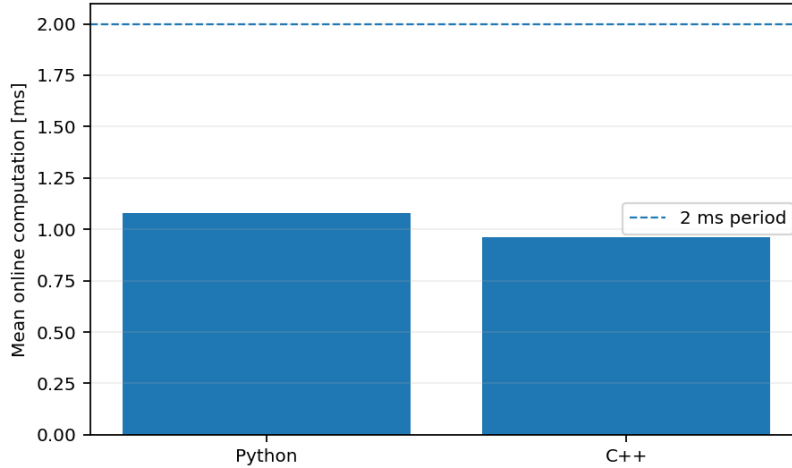


Figure 8: Mean online computation for the final optimized Python and C++ implementations. Both are below the 2 ms benchmark period on the measured system.

The C++ timing result was:

Metric	C++ result
Mean	0.9605 ms
P99	0.9732 ms
Maximum	9.9545 ms
Mean fraction of 2 ms period	48.02%

The mean and P99 measurements fit within a 2 ms period, supporting the decision to proceed to a 500 Hz Isaac Sim trial. The approximately 9.95 ms maximum outlier prevents a hard-real-time claim. These measurements were obtained on a general-purpose Linux environment and should be interpreted as implementation benchmarking, not deadline certification.

10 Engineering Interpretation

The final nominal MPC result supports several conclusions. First, a fixed local linear prediction model can track this moderate nonlinear UR5e benchmark accurately when combined with trajectory-dependent gravity feedforward. Second, prediction horizon cannot be selected from tracking error alone because computational cost grows strongly with optimization dimension. Third, reducing the control horizon provided a useful dimensionality reduction with only a small change in the tested terminal metric. Fourth, explicit constraints add structural capability even though they remain inactive in this nominal trajectory. Finally, implementation architecture is part of controller design: a mathematically identical QP changed from impractical at 500 Hz to sub-millisecond mean C++ execution after fixed and reference-known terms were moved offline.

The comparison with LQR is also instructive without assigning an overall winner. The final LQR produced lower overall joint RMSE (0.01550° versus 0.04847°) and lower orientation RMSE (0.03776° versus 0.12977°), while the MPC produced slightly lower TCP position RMSE (0.378 mm versus 0.419 mm). Under the nominal trajectory, the MPC constraints were inactive, so its main structural advantage—explicit constrained prediction—was not exercised. The subsequent robustness and simulation stages are therefore necessary before drawing broader controller conclusions.

11 Limitations and Reproducibility Boundaries

- The prediction model is one fixed linearization about the nominal start equilibrium; it is not nonlinear MPC and is not gain-scheduled.
- Gravity feedforward and the offline reference-gradient sequence use the known nominal reference. If the reference changes online, the reference-dependent term must be recomputed for the new horizon.
- The plant remains the ideal fixed-base rigid-body model used throughout Part 1B; friction, compliance, motor electrical dynamics, sensor noise, and the proprietary UR servo are excluded.
- The adopted torque limits are simulation constraints, not independently verified physical motor-torque ratings.
- Constraint inactivity in the nominal benchmark does not validate constrained behavior under saturation.
- Python–C++ agreement validates implementation consistency, not independent physical-model accuracy.
- Mean and P99 execution below 2 ms do not establish hard-real-time behavior because timing outliers above the deadline were observed.

12 Reproducibility Map

Purpose	Script / artifact
MPC implementation	controllers/mpc.py
Nominal nonlinear benchmark	analysis/mpc_nominal_benchmark.py
Computation profiling	analysis/mpc_computation_profile.py
Tolerance study	analysis/mpc_tolerance_sweep.py
C++ data export	analysis/export_mpc_cpp_data.py
Binary export	analysis/export_mpc_cpp_binary.py
C++ controller / cross-validation	cpp/mpc_controller.cpp
Nominal results	results/mpc/nominal
C++ interchange data	results/mpc/cpp_data

13 Conclusion

The MPC study began as a conventional constrained linear predictive-control implementation and became primarily an exercise in separating design mathematics from runtime computation. The selected $N_p = 160$, $N_c = 40$, $\rho_u = 10$ controller achieved sub-degree joint errors by a wide margin on the nonlinear nominal benchmark while remaining far from the adopted torque limits. More importantly, profiling and algebraic restructuring reduced the mean online computation from approximately 17.97 ms to 1.07–1.08 ms in Python, and the C++ implementation reduced it further to 0.960 ms while reproducing the Python torque sequence to approximately 2×10^{-10} N m.

The nominal design is therefore sufficiently characterized to leave controller tuning and proceed to the next question: how much of this performance is retained when the controller model no longer matches the plant and when external disturbances are introduced. That robustness study should precede the final Isaac Sim comparison.